What's a Tactic?

Thomas Sewell



May 28, 2025

This Talk

This talk is about the question "what is a proof tactic?"

- What is a proof?
- What is a proof tactic?
- Why is a tactic usually a functional program?
- Why do they seem so . . . arcane?

Some Proofs

Let's start by eyeballing some example proofs.

Some Proofs

Let's start by eyeballing some example proofs.

. . . take it away, Will Hunting.

```
lemma receive_ipc_step_invs:
  "⦃invs and st_tcb_at active thread and
            cte_wp_at ((=) NullCap) (thread, tcb_cnode_index 3) and
            (λs. ∀r∈zobj_refs cap. ex_nonz_cap_to r s) and
            K (next = Schedule_Next)⦄
        receive_ipc_step thread cap is_blocking next
    ⦃λnext. invs⦄"
  apply (rule hoare_gen_asm)
  apply (simp add: receive_ipc_step_def)
  apply (wpsimp wp: complete_signal_invs get_simple_ko_wp
                    Finalise_AI.gbn_wp)
  apply (frule st_tcb_ex_cap, clarsimp+)
   apply auto[1]
  apply (clarsimp simp: st_tcb_at_tcb_at)
  done
```

```
lemma message_protocol_invs_receiver_unblock:
  "{invs and K (epptr ≠ None) and
       obj_at (λko. ∃q. ko = Endpoint (RecvEP (r
    message_protocol sender receiver epptr
       (Sync_Receiver_Unblock payload) next
 {λnext. invs}"
 apply (simp add: message_protocol_def)
 apply (wpsimp wp: dxo_wp_weak)
        apply (simp add: invs_def valid_state_def valid_pspace_def)
        apply (wpsimp wp: sts_only_idle valid_irq_node_typ)+
      apply (wp gts_wp)
     apply (strengthen st_tcb_weakenE[mk_strg I A])
     apply (strengthen imp_consequent[where Q="st_tcb_at _ _ _"])
     apply (simp split: thread_state.split)
     apply (wp hoare_drop_imps valid_irq_node_typ)
    apply (wpsimp wp: get_simple_ko_wp split_del: if_split)+
 apply (simp split: endpoint.split_asm split del: if_split)
 apply (clarsimp simp: invs_def valid_state_def valid_pspace_def
                 obj_at_def
       split del: if_split)
 apply (strengthen reply_cap_doesnt_exist_strg)
 apply (erule(1) valid_objsE)
 apply (clarsimp simp: ep_redux_simps valid_obj_def valid_ep_def
       split del: if_split)
 apply (subgoal_tac "ko_at ko (the epptr) s" for ko)
  prefer 2
  apply (simp add: obj_at_def)
 apply (frule(1) sym_refs_ko_atD)
 apply (subst delta_sym_refs, assumption)
   apply (clarsimp simp: split: if_split_asm)
  apply (clarsimp simp: st_tcb_at_refs_of_rev split: if_split_asm)
    apply (clarsimp simp: st_tcb_at_def obj_at_def)
   apply (frule(1) sym_refs_st_tcb_atD)
   apply clarsimp
   apply (clarsimp simp: tcb_bound_refs_def split: option.split_asm)
  apply auto[1]
 apply (clarsimp simp: st_tcb_at_refs_of_rev)
 apply (intro conjI impI)
    apply (auto elim: st_tcb_ex_cap)[1]
   apply (erule(1) if_live_then_nonz_capD)
   apply clarsimp
   apply (simp add: live_def split: list.split_asm)
  apply (clarsimp dest!: st_tcb_at_idle_thread)
 apply (clarsimp elim!: st_tcb_weakenE)
 done
```

A Spectrum

We have some kind of a spectrum of proof detail.

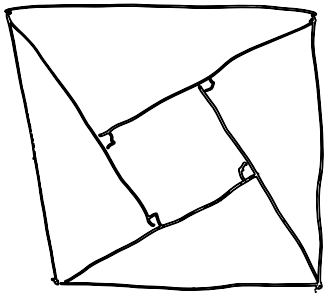
A Spectrum

We have some kind of a spectrum of proof detail. It's awful at both ends.

A Spectrum

We have some kind of a spectrum of proof detail. It's awful at both ends.

What about a classic?





$$a^2 = (b-c)^2 + 4\left(\tfrac{1}{2}bc\right)$$

$$= b^2 - 2bc + c^2 + 2bc$$

$$= b^2 + c^2$$

Formal Mathematical Proofs

We've all seen classical blackboard mathematics, manipulating a collection of figures placed in various positions. What do these operations *mean*?

- This issue came to a head in mathematics in the 1800s.

Let's look at the opposite approach, fully formal mathematics.

- Mostly invented by 19th century mathematicians.

Formal Mathematical Systems

Key ingredients of a formal mathematical system:

- A syntax of terms.
- A collection of proof rules.
- $\therefore$ A collection of provable terms.

For instance, to use a quantified proven fact, instantiation:

$$\forall x.\ P\ x \Longrightarrow P\ 12$$

And to use a proven implication, *modus ponens*:

$$P, (P \longrightarrow Q) \Longrightarrow Q$$

Context

Suppose we want to prove an implication:

$$P \longrightarrow Q$$

We want to use P multiple times in the proof of Q, which has many steps.

$$P \vdash Q \Longrightarrow P \longrightarrow Q$$

$$\Gamma, P \vdash Q \Longrightarrow \Gamma \vdash P \longrightarrow Q$$

$$\Gamma \vdash P \text{ if } P \in \Gamma$$

Adding a context makes it easier to "locally know" P, and fetch it as needed. This gives us proof *states*, configurations in which we can take steps and eventually derive interesting conclusions.

Context

Suppose we want to prove an implication:

$$P \longrightarrow Q$$

We want to use P multiple times in the proof of Q, which has many steps.

$$P \vdash Q \Longrightarrow P \longrightarrow Q$$

$$\Gamma, P \vdash Q \Longrightarrow \Gamma \vdash P \longrightarrow Q$$

$$\Gamma \vdash P \text{ if } P \in \Gamma$$

Adding a context makes it easier to "locally know" P, and fetch it as needed. This gives us proof *states*, configurations in which we can take steps and eventually derive interesting conclusions.

This context unstructured, an alternative would be to name each element.

Context and Curry

Does the context element Γ from the previous slide look familiar?

$$\Gamma, (x : \sigma) \vdash \text{expr} : \tau \implies \Gamma \vdash (\lambda x.\ \text{expr}) : (\sigma \Rightarrow \tau)$$

This is connected to the Curry-Howard isomorphism, which many of us have heard of. Proof checking in a simple logic is structurally equivalent to type-checking for a simple λ-calculus.

Curry-Howard

We could give an fp-syd length talk on the Curry-Howard isomorphism, setting up a logic whose proof terms are a λ language whose type system is the logic.

I don't want to go through this in detail now. I'm not personally a huge fan of the dependently typed approach. If you're interested, we can link you to other slide decks or talk occasions.

Let's think about some high-level intuitions though:

- What structural type would you give to the collection of proofs of $P \vee Q$? Or $P \wedge Q$?
- What does the typing process for a *recursive* function that consumes a natural number look like?
  - Note that it is essential our functions are terminating.

The FP/Logic Link

We've seen one link between functional programming and formal logic: the Curry-Howard isomorphism.

It's not the only example. The history of computer science has connected them together.

Some Examples

- Isabelle/HOL
  - Implemented in SML
  - `type tactic = thm -> thm seq`
- HOL4
  - Implemented in SML
  - `type tactic = goal -> goal list * validation`
- Rocq/Coq
  - Curry-Howard
  - Implemented in OCaml
  - Term language and proof language are Gallina
  - No single tactic language (built-in, Ltac, etc)
- Lean
  - Curry-Howard
  - Lean*
  - Lean†

The Grandparent of all the Examples

Why is the FP crowd so inter-mingled with the proof crowd?

- Edinburgh LCF
  - Implemented in ML
  - Invented ML

This is one of the cool things about the CakeML  project, that it builds a prover in a language in a prover in this style.

The Grandparent of all the Examples

Why is the FP crowd so inter-mingled with the proof crowd?

- Edinburgh LCF
  - Implemented in ML
  - Invented ML

This is one of the cool things about the CakeML  project, that it builds a prover in a language in a prover in this style. The LCF style.

Let's look at the big idea of LCF.

Elaboration and LCF

We've seen a spectrum of formality in mathematics. In the most informal style, we highlight some key sub-terms and transform our expressions, and the reader understands our justification. In the most formal style, we have only primitive combinations and substitutions.

It is convenient to give the user an *assistant*, a computer program that elaborates the user's guidance into a completed formal proof.

The user wants to provide minimal guidance. What is the absolute minimum? Are we familiar with Kolmogorov complexity?

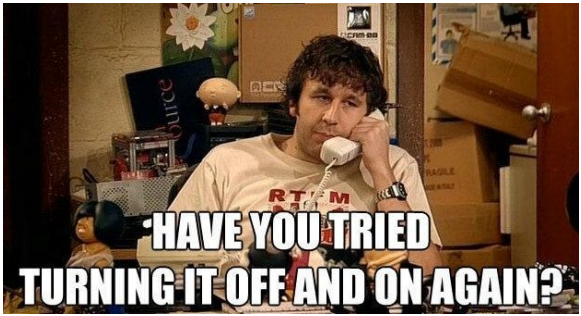
In the **LCF** style, the proof document is a program and the prover/checker is just a library.

- Some **kernel** types are protected by the type system.
  - e.g. every **thm** is a theorem of the logic
- Design by Robin Milner in implementation of Edinburgh LCF.
- Led to the ML programming language family.

Do we need FP for LCF?

LCF style: a proof is a program and the prover is a library.

We need the type system to prevent unauthorized construction of theorems. We also want our proof scripts to be predictable. Functional languages are a good fit for this, but are they the only option?



It might be possible to use a language like Rust, with loops and semicolons, but with effective control on the side-effects of the program.

Typical Tactics

In HOL4:

```
type tactic = goal -> goal list * validation
type goal = term list * term
type validation = thm list -> thm
```

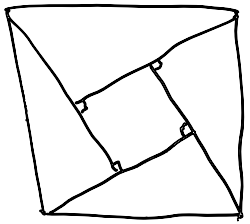
- Note that `thm` objects can have assumption lists.

Standard combinators:

- `THEN :  tactic * tactic -> tactic`
- `ORELSE :  tactic * tactic -> tactic`
- `ALL_TAC :  tactic`
- `NO_TAC :  tactic`
- `THEN1 :  tactic * tactic -> tactic`

See `src/1/Tactic.sig` and `src/1/Tactical.sig` for more.

Typical Tactic Concepts



$$a^2 = (b-c)^2 + 4\left(\tfrac{1}{2}bc\right)$$
$$= b^2 - 2bc + c^2 + 2bc$$
$$= b^2 + c^2$$

How do we tell the system "we want to cancel $2bc$"?

- Provide the whole context.
- Provide the focal term $2bc$.
- Provide a pattern that matches the focal term $2 * _ * _$.
- Ask for cancellation in general.
- Ask for ring-algebra related simplification in general.
- Ask for simplification in general.

Typical Tactic Concepts II

$$a^2 = b^2 - 2 * b * c + c^2 + 2 * b * c \vdash a^2 = b^2 + c^2$$

A very common approach: provide a pattern by providing a rule.

- Produce the rule $\forall x\ y\ z.\ x - y + z = x + z - y$.
- Specialise y to $2 * b * c$ to get $x - 2bc + z = x + z - 2bc$.
  - $\forall x\ b\ c\ z.\ x - 2bc + z = x + z - 2bc$.
- Match this against the goal: $b^2 - 2bc + (c^2 + 2bc) = b^2 + (c^2 + 2bc) - 2bc)$.
- Use a theorem for applying equalities in a wider context.
- Progress! Now we need to reassociate.

Typical Tactic Concepts II

Reassociating to right-associated:

$$\forall a\ b\ c.\ (a + b) + c = a + (b + c)$$

We can read the above theorem as (a) a way of proving a goal that looks exactly like its conclusion, or (b) as a rewrite of any sub-term that matches its left-hand-side.

Option (b) is useful for re-associating our whole problem.

$$\forall x\ y\ z.\ x > 0 \longrightarrow y \geq z \longrightarrow x * y \geq x * z$$

We can read the above theorem as (a) a way of using assumptions of the form $x > 0$ or (b) a way of proving inequalities with a shared multiplicative factor. Option (b) sounds more useful.

Hobby-Horse

Without going into too many details, we sometimes have the option to describe a "move" on a proof state in two different ways.

Option 1:

```
conditional-rewrite-with thmA
THEN conjunction-tac
THEN1 intro-rule thmB
```

Option 2:

```
let monsterTheorem = COMPOSE thmA [thmB]
conditional-rewrite-with monsterTheorem
```

I have a strong personal preference for option 2, when it is possible. The key advantage is that you can examine the composed theorem.

Stronger Tactics

This is all very low-level.

What about a stronger tactic that can solve whole logical puzzles?

- This goal is a solvable as a satisfiability (SAT) problem.
- This goal is a solvable with a standard inequality solver.
- This component of this goal is provable via the above, and can be replaced with True.

These kinds of tools are available in nearly all proof assistants. They don't change the world, however, because of the problem of under-defined terms.

Much Stronger Tactics

Can we throw the tactic language away, and just appeal to a strong solver?

For instance, SAT/SMT, which I gave an fp-syd talk about maybe 10 years ago?
ITPs vs ATPs (interactive vs automated):

- Day 1 experience is completely different.
- The tools tend to converge.
  - Interactive tools get automation.
  - Automated tools get fine-grained or interactive debuggers.

Much Stronger Tactics

Can we throw the tactic language away, and just appeal to a strong solver?

For instance, SAT/SMT, which I gave an fp-syd talk about maybe 10 years ago?
ITPs vs ATPs (interactive vs automated):

- Day 1 experience is completely different.
- The tools tend to converge.
  - Interactive tools get automation.
  - Automated tools get fine-grained or interactive debuggers.

Also, Abstract Interpretation & Model Checking.

- Big research agenda from the previous FM generation.
- Built a collection of decidable domains (a bit like theories).
- The idea was to meet in the middle with programming.
  - analysis for more domains $\longleftrightarrow$ programming within those domains
- This never resulted in an ideal environment.

Time to Finish Up

We've hit the limit of how many slides I can write/cannibalize while eating pizza.

Tactics! We need them, but we suffer through them. I haven't had enough time to tell you all about them, but hopefully you've learned something

Questions?

What is a logical proof?

What is a logical proof?

What is a formal proof?

Why would we want to computerise them?

What is a logical proof?

What is a logical proof?

What is a formal proof?

Why would we want to computerise them?

- Because we're computer programmers.

What is a logical proof?

What is a logical proof?

What is a formal proof?

Why would we want to computerise them?

- Because we're computer programmers.
- Mathematicians have reasons too.

A Potted History of Logic in Mathematics

- $\approx$ 300BC: Euclid's axioms and proofs.
- 1800s: A huge explosion of mathematics, and various crises.
  - Calculus, limits, etc sometimes lead to mistakes & contradictions.
- Late 1800s:
  - Formal presentation of logic.
  - Set theory & Principia Mathematica.
- $\approx$ 1900: The Hilbert program
  - The aim is to develop an ideal logic.
  - Settle every question, and compute every answer.
- 1930: Gödel's incompleteness theorem & Turing's version.
- 1980s on: Formalisation of mathematics in ITPs.

Formal Proofs: The Future of Mathematics?

- 1611: Kepler conjecture is stated.
  - The ideal packing of spheres is the greengrocer or cannonball packing.
- 1930s: Early Ramsey number results.
  - Every blue/red graph with more than $R(r, s)$ nodes contains either r blue-connected or s red-connected nodes.
  - e.g. The Erdös-Szekeres theorem.
- 1998: Hales and Ferguson give a computer-aided proof of Kepler.
  - The Annals of Mathematics take 5 years to make a decision.
- 2003-2014: Hales leads the Flyspeck project.
  - Replaying the Kepler proof, mostly using HOL-Light,
- 2023: New upper-bound progress on Ramsey numbers.
  - Bhavik Mehta publishes a Lean proof after 6 months.

Formal Proofs: The Future of Language Research?

Recent academic results in programming language design are likely to come with proofs, and those proofs are likely to be formalised using a theorem prover.

- POPLmark effort 2005-2012